

COM106: Introduction to Databases

SQL – Queries

COM106 Weekly Schedule - Lecture & Lab Delivery

Week 9 Lab is the in-class exam (Set Exercise, 90-minute class test, 40% of module mark).

Week	Lecture	Lab / Seminar
1	Module Introduction & DB Basics and Security	Lab 1: Getting Started with MYSQL
2	Introduction to Entity-Relationship Modelling	Lab 2: Creating ER Diagrams with draw.io
3	ER Modelling 2 & Mapping to the Relational Model	Lab 3: ER Diagram Design from Scenarios
4	Normalisation	Lab 4: Normalisation Practice
5	Introduction to SQL (DDL, DML, DCL)	Lab 5: Building a Database with SQL (businessdb)
6	**SQL Queries: Single-Table & Multiple-Table** - SELECT structure & WHERE logic - ORDER BY & DISTINCT - Aggregates, GROUP BY & HAVING - Joins: WHERE vs INNER JOIN - Nested queries & self joins - Views	**Lab 6: SQL Retrieval Exercises (projemp)** - The projemp schema - GROUP BY & HAVING exercises - DISTINCT & counting - Multi-table join queries - Mixed retrieval practice

https://github.com/KodZone/COM106-MYSQL



Recap

- **SQL components:** DDL defines database structures, DML retrieves and changes data, and DCL manages permissions.
- **Creating tables:** Use `CREATE TABLE` with suitable data types and primary keys, including composite keys.
- **Integrity constraints:** Apply `NOT NULL`, `DEFAULT`, `UNIQUE` and `CHECK` to enforce data rules.
- **Linking tables:** Foreign keys establish relationships and maintain referential integrity.
- **Changing structures:** Use `ALTER TABLE` to add, modify or remove columns and constraints.
- **Managing records:** Use `INSERT`, `UPDATE` and `DELETE`, with `WHERE` to target specific records.



Recap - Query is for Read Records

- So far, we've seen a number of **CRUD** operations:
  - **C** – Create,
  - **R** – Read,
  - **U** – UPDATE,
  - **D** – DELETE.
- Now look at **R** operations – the **SELECT** statement – used to retrieve data/information from the database

Single Table vs. Multiple Tables



General Form of SELECT

SELECT *<attribute list>*
FROM *<table list>*
WHERE *<selection clause>*
    *[**AND/OR/NOT** <selection clause>….]*
GROUP BY *<attribute list>* ***[grouping]***
HAVING *<aggregate function selection>* ***[between groups]***
ORDER BY *<attribute list>* ***[sorting]***

Natural language query:

Get the names and salaries of all employees who are on the first floor.

SQL query:

```
SELECT ename, salary
FROM EMPLOYEE
WHERE floor = 1 ;
```

SELECT always returns a table
- even if a single value is returned..

May want information returned from more than one table
List all tables used in the query.

General Form of SELECT

How to Construct an SQL SELECT Query

Need to interpret the meaning of the **natural language query** (sentence) to find:

1. What columns do we want to see in the query result table?
   (Which **attributes** (columns) go into the **SELECT** clause?)
2. What other **attributes** are involved in the query
   (for selecting in a **WHERE/AND/OR/NOT** clause)
   or grouping **(GROUP BY** clause)
   or choosing between groups (**HAVING** clause)
   or sorting (**ORDER BY** clause)
3. What table(s) are required to get all the attributes involved in the **SELECT** clause or from the **WHERE**, **GROUP BY**, **HAVING** or **ORDER BY** clauses
   (all the tables used will be included in the **FROM** clause)

For example: ***Get the names and salaries of all employees who are on the first floor***

1. Attributes in **SELECT** clause:
   We want the names of the employees (**ename**)
   and the salaries of the employees (**salary**)
2. The **WHERE** clause attributes:
   We need to find employees on the first floor (**floor = 1**)
3. Table(s) with required attributes in **FROM** clause:
   ename, **salary** and **floor** all come from the **EMPLOYEE** table

How to Join Multiple Tables

Often, the information required is spread across **two or more tables**.

ename *and* **floor** *are in the* **EMPLOYEE** *table*

For example - ***Get the names of employees and the roles they have performed for employees on the 1st floor.***

role *is in the* **WORKS_ON** *table*

Retrieving data from two or more tables requires **defining the relationship between different columns across the tables in the query** (normally **across PK/FK matches**)

There are **TWO** SQL approaches

- **Original** (**Mark 1**): specify join clause as an extra **WHERE/AND** clause
- **New** (**Mark 2**): specify **INNER JOIN** clause in a subsidiary **ON** clause after FROM clause

Join Multiple Tables

Get the names of employees and the roles they have performed for employees on the 1st floor.

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

1 Original (Mark 1) – Using WHERE/AND Clause (Old Style)

- This is the **traditional** (older) way of writing joins.

```
SELECT ename, role
FROM EMPLOYEE, WORKS_ON
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND floor = 1;
```

2 New (Mark 2) – Using JOIN ON Clause (Modern SQL)

- This is the **standard SQL-92** approach.

```
SELECT ename, role
FROM EMPLOYEE
INNER JOIN WORKS_ON ON EMPLOYEE.enum = WORKS_ON.enum
WHERE floor = 1;
```

Join Multiple Tables

Mark 2 Joins - Alternative method for SQL joins

1. all relations participating in the join are listed in the FROM clause
2. Use of **INNER JOIN** as the join operator
3. join clauses must be specified in a subsidiary **ON** clause

*There are a number of different **JOIN** operators – see later*

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

e.g. ***Get the names and salaries of employees who work on projects led by 'Gates'***

Mark 1

```
SELECT ename, salary
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND WORKS_ON.pnum = PROJECT.pnum
AND leader = 'Gates';
```

Mark 2

```
SELECT ename, salary
FROM EMPLOYEE
INNER JOIN WORKS_ON ON EMPLOYEE.enum = WORKS_ON.enum
INNER JOIN PROJECT ON WORKS_ON.pnum = PROJECT.pnum
WHERE leader = 'Gates';
```

Case Study Schema

In the examples that follow the EMPLOYEE, PROJECT, WORKS_ON database is used -

with relational schema and sample data as sown:

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

PROJECT

pnum	pname	leader
121	IT	Gates
135	Design	Sinclair
147	Analysis	Einstein
216	Publicity	Saatchi
227	Theatre	Dench
251	Sport	Shearer

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager

Basic Query Statement

Basic Query – Single Table

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

So, the **natural language** query: ***Get the names and salaries of all employees who are on the first floor***

can be answered using an **SQL** query:

```
SELECT ename, salary
FROM EMPLOYEE
WHERE floor = 1 ;
```

Names and salaries.

All attributes are in the EMPLOYEE table.

On the first floor.

EMPLOYEE

enum	ename	salary	floor
852341	***Smith***	***15000***	***1***
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	***Adams***	***30500***	***1***
852514	Doyle	11650	2
852530	Evans	26980	4



Result Table

ename	salary
Smith	15000
Adams	30500

Get the employee details of all employees on the first floor

```
SELECT *
FROM EMPLOYEE
WHERE floor = 1 ;
```

** - all attributes*

Result Table

enum	ename	salary	floor
852341	Smith	15000	1
852491	Adams	30500	1

SELECT *

is equivalent to

SELECT enum, ename, salary, floor

Basic Query – Multiple Tables

Natural Language Query: ***Get the names of employees and the roles they have performed.***

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

```
SELECT ename, role
    FROM EMPLOYEE, WORKS_ON
    WHERE EMPLOYEE.enum = WORKS_ON.enum;
```

Mark 1 Join

*Join on PK/FK match – **EMPLOYEE.enum (PK)** and **WORKS_ON.enum (FK)***

```
SELECT ename, role
FROM EMPLOYEE
INNER JOIN WORKS_ON ON EMPLOYEE.enum =
WORKS_ON.enum;
```

Mark 2 Join

Result Table

Ename	role

EMPLOYEE

enum	ename	salary	floor
852341	*Smith*	15000	*1*
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	*Adams*	30500	*1*
852514	Doyle	11650	2
852530	Evans	26980	4

WORKS_ON

enum	pnum	role
852341	121	*Manager*
852341	135	*Designer*
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	*Designer*
852491	216	*Manager*
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager

More Examples of Basic Query

- ✅ **Retrieve all employees details.**

```
SELECT *
FROM EMPLOYEE;
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

- ✅ **Retrieve the names of employees and the project_number involved**

```
SELECT ename, pnum
FROM EMPLOYEE, WORKS_ON
WHERE EMPLOYEE.enum = WORKS_ON.enum;
```

Mark 1

```
SELECT ename, pnum
FROM EMPLOYEE
JOIN WORKS_ON ON EMPLOYEE.enum = WORKS_ON.enum;
```

Mark 2

More Examples of Basic Query

- ✅ Retrieve the names of **employees** and the name of projects involved.

```
SELECT ename, pname
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND WORKS_ON.pnum = PROJECT.pnum;
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Mark 1

```
SELECT ename, pname
FROM EMPLOYEE
JOIN WORKS_ON ON EMPLOYEE.enum = WORKS_ON.enum
JOIN PROJECT ON WORKS_ON.pnum = PROJECT.pnum;
```

Mark 2

Basic Query with Operators

Basic Query with Operators – Single Table

WHERE Clauses with an AND

Get the employee numbers for employees in project 121 who have an Assistant role

```
SELECT enum
FROM WORKS_ON
WHERE pnum = 121
AND role = 'Assistant' ;
```

Result Table

enum
852455
852514

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	***121***	***Assistant***
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	***121***	***Assistant***
852514	216	Consultant
852514	251	Manager
852530	147	Manager

Alternative WHERE Clauses with an OR

Get the name of employees working on the 1st or 2nd floor

```
SELECT ename
FROM EMPLOYEE
WHERE floor = 1
OR floor = 2 ;
```

EMPLOYEE

enum	ename	salary	floor
852341	***Smith***	***15000***	***1***
852358	Jones	19000	3
852407	Brown	16000	3
852455	***White***	***25100***	***2***
852491	***Adams***	***30500***	***1***
852514	***Doyle***	***11650***	***2***
852530	Evans	26980	4

Result Table

ename
Smith
White
Adams
Doyle

Basic Query with Operators – Single Table

Other Useful Operators with the WHERE Clause

Using the **IN** Operator to Specify Multiple Values in a WHERE Clause.

```
SELECT ename
FROM EMPLOYEE
WHERE floor IN (1, 2);
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Using the **BETWEEN** Operator to Select Values within a Range

```
SELECT ename
FROM EMPLOYEE
WHERE ename BETWEEN 'Adams' AND 'Evans';
```

values can be numbers, text, or dates.

The **NOT** Operator

Get the names of employees on all floor, except floors 1 and 3

```
SELECT ename
FROM EMPLOYEE
WHERE floor NOT IN (1, 3);
```

Combining Operators in the WHERE Clause.

Operators can be combined to form more complex queries, using parenthesis as necessary to establish precedence.

Get details of all projects between project number 121 and 227, except those led by Sinclair or Saatchi

```
SELECT *
FROM PROJECT
WHERE pnum BETWEEN 121 AND 227
AND leader NOT IN ('Sinclair', 'Saatchi');
```

Basic Query with Operators – Multiple Tables

Natural Language Query: ***Get the names of employees and the roles they have performed for employees on the 1st floor.***

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

```
SELECT ename, role
FROM EMPLOYEE, WORKS_ON
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND floor = 1;
```

Mark 1 Join

Join on PK/FK match – EMPLOYEE.enum (PK) and WORKS_ON.enum (FK)

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

Result Table

ename	role
Smith	Manager
Smith	Designer
Adams	Designer
Adams	Manager

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager

Basic Query with Operators – Multiple Table

Natural Language Query: ***Get the names and salaries of employees who work on projects led by 'Gates'***

*Requires attributes from all **three** tables*

```
SELECT ename, salary
FROM EMPLOYEE, WORKS_ON, PROJECT
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND WORKS_ON.pnum = PROJECT.pnum
AND leader = 'Gates';
```

*Mark 1 Join on **PK/FK match***

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4

Result Table

ename	salary
Smith	15000
White	25100
Doyle	11650

WORKS_ON

enum	pnum	role
852341	121	Manager
852341	135	Designer
852358	147	Consultant
852358	135	Consultant
852407	216	Assistant
852455	121	Assistant
852455	227	Manager
852491	135	Designer
852491	216	Manager
852514	121	Assistant
852514	216	Consultant
852514	251	Manager
852530	147	Manager

PROJECT

pnum	pname	leader
121	IT	Gates
135	Design	Sinclair
147	Analysis	Einstein
216	Publicity	Saatchi
227	Theatre	Dench
251	Sport	Shearer

WORKS_ON.*pnum* = PROJECT.*pnum*

Query with Aggregate Functions, and GROUPED BY and HAVING Clauses

Query with Aggregate Functions – Single Table

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Aggregate Functions

SQL provides a number of built-in aggregate functions that return a **single** value, calculated from values in a column.

SUM( )	Returns the Total Value
AVG( )	Returns the Average Value
MAX( )	Returns the Highest Value
MIN( )	Returns the Lowest Value
COUNT( )	Returns the number of tuples (rows) in result table

Operate only on numeric attributes

Aggregate functions can **ONLY** appear in **SELECT** and **HAVING** clauses

What is the total salary of all employees?

```
SELECT SUM (salary)
FROM EMPLOYEE;
```



Result
144230

Aggregate functions normally appear alone in a SELECT clause (except for the special case where a GROUP BY clause is present)

Query with Aggregate Functions – Single Table

EMPLOYEE (<u>enum</u>, ename, salary, floor)
PROJECT (<u>pnum</u>, pname, leader)
WORKS_ON (<u>enum</u>*, <u>pnum</u>*, role)

Aggregate Functions

What is the average salary for employees on the 3rd floor?

```
SELECT AVG (salary)
FROM EMPLOYEE
WHERE floor = 3 ;
```



Result
20604

** - can be used ONLY with COUNT*
*COUNT using PK or * - other attributes may be NULL.*

How many employees are on the 1st floor?

```
SELECT COUNT (*)
FROM EMPLOYEE
WHERE floor = 1;
```

or

```
SELECT COUNT (enum)
FROM EMPLOYEE
WHERE floor = 1;
```



Result
2

Query with Aggregate Functions – Multiple Table

- Average Salary of Employees on Floor 3 Who Work on Projects

```
SELECT AVG(salary)
FROM EMPLOYEE, WORKS_ON
WHERE EMPLOYEE.enum = WORKS_ON.e
AND floor = 3;
```

Mark 1

```
SELECT AVG(salary)
FROM EMPLOYEE
JOIN WORKS_ON ON EMPLOYEE.enum = WORKS_ON.enum
WHERE floor = 3;
```

Mark 2

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

GROUP BY Clause in SQL – Explanation & Examples

- The **GROUP BY clause** in SQL is used to group rows that have the same values in specified columns.
- It is often **used with aggregate functions** (SUM(), AVG(), COUNT(), MIN(), MAX()) to summarize data.

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

- The attribute used in the GROUP BY clause **must appear** in the SELECT clause
- **Don't use more than one attribute** in the GROUP BY clause (unless you intend to use grouping and subgrouping – not covered in these notes)

GROUP BY Clause in SQL

- **The GROUP BY Clause -** Groups result table by attribute(s).
- **GROUP BY** clauses group the data **by value** using a field that **MUST** therefore have duplicate values so that at least some groups have multiple records.
- An aggregate function can then be applied to each group.
- All simple attributes appearing in the SELECT clause **MUST** appear in the GROUP BY clause

e.g. ***Get the average salary by floor of all employees***

```
SELECT floor, AVG (salary)
FROM EMPLOYEE
GROUP BY floor ;
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

EMPLOYEE

enum	ename	salary	floor
852341	Smith	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	Adams	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4



Result Table

floor	AVG (salary)
1	22750
2	18375
3	17500
4	26980

GROUP BY Clause in SQL

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

The HAVING Clause

- Can **ONLY** be used when a **GROUP BY** clause is present - **used to choose between groups**.
- Can **ONLY** contain expressions using **aggregate functions**

e.g. ***Get the floors having two or more employees***

```
SELECT floor
FROM EMPLOYEE
GROUP BY floor
HAVING COUNT(*) >= 2 ;
```



Result Table

floor
1
2
3

- HAVING clauses are used to choose **between** groups and **cannot** appear **without** a preceding GROUP BY clause.
- HAVING clauses **MUST** be constructed as:

Aggregate function	*compared with*	*a value*
count( ), avg( ), etc	**=, <, >, etc**	**number**

Query with ORDERD BY Clause and DISTICT Clause

Query with ORDERD BY Clause

- The **ORDER BY clause** in SQL is used to sort the result set based on one or more columns, in either **ascending (ASC)** or **descending (DESC)** order.

```
SELECT column_name(s)
FROM table_name
ORDER BY column_name [ASC | DESC];
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

e.g. ***List the employee names in alphabetic order***

```
SELECT ename
FROM EMPLOYEE
ORDER BY ename ASC;
```



***ASC** is the **default** value and thus not strictly necessary in this example*

ename
Adams
Brown
Doyle
Evans
Jones
Smith
White

4 ORDER BY with Aggregation

✅ Example: Show Total Salary Per Floor, Ordered from Highest to Lowest

```
SELECT floor, SUM(salary) AS total_salary
FROM EMPLOYEE
GROUP BY floor
ORDER BY total_salary DESC;
```

```
SELECT floor, SUM(salary)
FROM EMPLOYEE
GROUP BY floor
ORDER BY SUM(salary) DESC;
```

✅ The **AS total_salary** is an **alias** that renames the result of SUM(salary) for better readability.

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Query with DISTINCT Clause

- The **DISTINCT clause** in SQL is used to **eliminate duplicate values** from a result set. It ensures that **only unique values are returned**.

```
SELECT DISTINCT column_name(s)
FROM table_name;
```

The DISTINCT Predicate

Eliminates duplicate tuples (rows)

e.g. ***Get a list of roles worked on (without repeat values)***

SELECT DISTINCT role
FROM WORKS_ON;

role
Manager
Designer
Consultant
Assistant
Administrator

DISTINCT clauses are most useful when used with an attribute which does NOT have unique values and the duplicated result rows are not required

Without DISTINCT, the query would return multiple entries for each role – as per the original WORKS_ON table

Nested Queries

Nested Queries (Subqueries) in SQL

- **A nested query**, also known as **a subquery**, is a **query inside another query**. The inner query runs first, and its result is used by the outer query.

```
SELECT column_name(s)
FROM table_name
WHERE column_name = (SELECT column_name FROM another_table WHERE condition);
```

- It can be used in **SELECT, WHERE, FROM, or HAVING** clauses.
- The SELECT clause contains attributes from only one relation + extra information is required from another table such as an aggregate function.

Nested Queries

Links between the query blocks can be achieved:

- using the **IN predicate** if multiple values may be returned from the lower clause
- using >,>=,<,<=,=,<> whenever only one value will be returned from the lower clause

e.g. ***Get the names of employees who have worked in the role of 'Consultant'.***

This query can be solved by using different approaches of SQL statements:

1) Mark 1: By using Mark 1 join (WHERE ..)
2) Mark 2: By using Mark 2 join (INNER JOIN .. ON ..)
3) Nested Query: By using query inside another query (WHERE .. (SELECT …)

Nested Queries

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Get the names of employees who have worked in the role of 'Consultant'.

As Mark 1 Join:

```
SELECT ename
FROM EMPLOYEE, WORKS_ON
WHERE EMPLOYEE.enum = WORKS_ON.enum
AND role = 'Consultant';
```

As Mark 2 Join:

```
SELECT ename
FROM EMPLOYEE
INNER JOIN WORKS_ON ON EMPLOYEE.enum
= WORKS_ON.enum
WHERE WORKS_ON.role = 'Consultant';
```

Result Table

ename
Jones
Jones
Doyle

As a Nested Query:

```
SELECT ename
FROM EMPLOYEE
WHERE enum IN
    (SELECT enum
    FROM WORKS_ON
    WHERE role = 'Consultant');
```

*The nested **SELECT** clause returns a **list of enum** of staff who are consultants*

So, WHERE clause is equivalent to:

WHERE enum IN (852358, 852514);

Nested Queries – Another Example

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

e.g. ***Get the employee names for employees earning salaries above the average salary for all employees***

```
SELECT ename
FROM EMPLOYEE
WHERE salary >
      (SELECT AVG (salary)
      FROM EMPLOYEE);
```

Result Table

ename
White
Adams
Evans

*The nested **SELECT** clause returns a **single value** – average salary*

***RECALL:** Aggregate functions can only appear in **SELECT** or **HAVING** clauses*

Nested Queries with Negation

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Nested Queries with Negation

e.g. ***Get all details of employees who do not work on project 121.***

```
SELECT *
FROM EMPLOYEE
WHERE enum NOT IN
        (SELECT enum
        FROM WORKS_ON
        WHERE pnum = 121);
```

The query is evaluated **backwards** as:

subquery: pass the **enums** of those employees working on project 121 to top query

top query: get the details of the employees **NOT IN** that list of **enums**

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

Multi-level Nested Queries

e.g. ***Get the names of employees working on projects that have no consultants working on them***

Get the names of employees
(According to the list of employees)
NOT working on
projects that have a consultant on them

Sometimes, it is helpful to **reframe** the query.

It can also be useful to build the query from the **bottom up.**

lowest subquery: Get the projects which have consultants working on them

middle subquery: Get the employee numbers for projects not in this list

top query: Get the employee names of employees from this employee list

Multi-level Nested Queries

Get the names of employees NOT working on projects that have a consultant on them

lowest subquery: Get the projects which have consultants working on them

```
SELECT pnum
FROM WORKS_ON
WHERE role = 'Consultant'
```

middle subquery: Get the employee numbers for projects not in this list

```
SELECT enum
FROM  WORKS_ON
WHERE pnum NOT IN
        (SELECT pnum
        FROM WORKS_ON
        WHERE role = 'Consultant')
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

top query: Get the employee names of employees from this employee list

```
SELECT DISTINCT ename
FROM EMPLOYEE
WHERE enum IN
        (SELECT enum
        FROM  WORKS_ON
        WHERE pnum NOT IN
                (SELECT pnum
                FROM WORKS_ON
                WHERE role = 'Consultant')) ;
```



Result Table

ename
Smith
White
Doyle

VIEWS

VIEWS in SQL

- A View in SQL is a **virtual table** that is based on the result of a SELECT query. It does not store data itself but provides a way to **simplify complex queries** and improve security by restricting direct access to certain columns or rows.
- It is only materialized (actually created as a table) at run time from the base table(s)
- Views can be manipulated mostly as though it were a base table (from user perspective)

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

VIEWS in SQL

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)

e.g. CREATE VIEW **FLOOR_1_EMP** (**employee_no**, **name**)
AS SELECT enum, ename
FROM EMPLOYEE
WHERE floor = 1;

Can give the attributes new names

Views can be used in queries, just like any other table

EMPLOYEE

enum	ename	salary	floor
852341	***Smith***	15000	1
852358	Jones	19000	3
852407	Brown	16000	3
852455	White	25100	2
852491	***Adams***	30500	1
852514	Doyle	11650	2
852530	Evans	26980	4



FLOOR_1_EMP

employee_no	name
852341	Smith
852491	Adams

VIEWS in SQL

- ✅ Example 2: Create a View for Employee Project Assignments

```
CREATE VIEW EmployeeProjects
AS SELECT ename, pnum, pname, role
FROM EMPLOYEE
JOIN WORKS_ON ON EMPLOYEE.enum = WORKS_ON.enum
JOIN PROJECT ON WORKS_ON.pnum = PROJECT.pnum;
```

EMPLOYEE (enum, ename, salary, floor)
PROJECT (pnum, pname, leader)
WORKS_ON (enum*, pnum*, role)




THE
END